Lab 1 Report

과제 개요 및 구현 과정

이번 과제는 Verilog를 사용하여 간단한 ALU 기능을 구현하는 과제다. Skeleton code의 ALU.v 파일의 내용을 수정하였다.

```
input [31:0]        operand1,
input [31:0]        operand2,
input [4:0]         shamt,
input [3:0]         funct,
output reg [31:0]   alu_result
```

2개의 32-bit 피연산자 `operand1`, `operand2`, 5-bit shift amount `shamt`, 그리고 4-bit의 연산자 `funct`를 입력으로 받아서 ALU에 집어넣고, ALU 연산 결과를 32-bit output register `alu_result`에 넣어주면 된다.

구현해야 하는 ALU 연산자는 다음과 같고, 간단한 연산자의 경우 어려움 없이 high-level language 방식을 차용하여 구현하였다.

```
case (funct)
    `ALU_ADDU: alu_result = operand1 + operand2;
    `ALU_AND : alu_result = operand1 & operand2;
    `ALU_NOR : alu_result = ~(operand1 | operand2);
    `ALU_OR  : alu_result = operand1 | operand2;
    `ALU_SLL : alu_result = operand2 << shamt;
    `ALU_SRA : alu_result = $signed(operand2) >>> shamt;
    `ALU_SRL : alu_result = operand2 >> shamt;
    `ALU_SUBU: alu_result = operand1 - operand2;
    `ALU_XOR : alu_result = operand1 ^ operand2;
    `ALU_SLT : alu_result = ($signed(operand1) < $signed(operand2)) ? 32'd1 : 32'd0; // Signed 비교
    `ALU_SLTU: alu_result = (operand1 < operand2) ? 32'd1 : 32'd0; // Unsigned 비교
    `ALU_EQ  : alu_result = (operand1 == operand2) ? 32'd1 : 32'd0;
    `ALU_NEQ : alu_result = (operand1 != operand2) ? 32'd1 : 32'd0;
    `ALU_LUI : alu_result = operand2 << 16; // LSB 16-bit에 있는 Immediate 값을 MSB 16-bit로 이동
endcase
```

몇 가지 연산자는 단순하게 `alu_result = operand1 funct operand2`의 간단한 방식으로 구현되지 않아, 생각의 과정을 거쳐 다음의 구현 방식을 사용하였다:

- ALU_SRA: Right arithmetic shift는 Logical shift와 구분되어 꺽쇠 3개(>>>)를 사용하고, sign bit를 고려해야 하기 때문에 `operand2` 앞에 signed를 씌워 signed 32-bit로 바꿔주었다.
- Set 연산자에서는 삼항 연산자를 사용하였다. 비교 연산의 결과에 따라 32-bit의 1 또는 0으로 설정된다.
- 마찬가지로 Equal, Not Equal에서도 비교 연산의 결과에 따라 설정해주는 삼항 연산자를 사용하였다.
- ALU_LUI: `operand2`가 immediate 값이 되어 16-bit left shift 연산을 적용하였다.

테스트

```
10000    0111
10001
```

```
// FIXME (Set the number of tests)
// Depending on the number of testcases you generate in the CPP code
parameter TEST_SIZE = 10000;
```

funct.mem 파일에 총 10,000개의 funct 코드가 있어 ALU_tb.v 파일의 테스트 사이즈를 수정해주었다.

```
HW01 Skeleton Code  iverilog ALU.v ALU_tb.v
HW01 Skeleton Code  ./a.out
```

과제 명세에 따라 Verilog 파일들을 컴파일하고 실행하였다.

```
TEST CTRL Sig: b :
PASSED
TEST CTRL Sig: a :
PASSED
TEST CTRL Sig: 7 :
PASSED
PASSED =        10000, FAILED =           0
ALU_tb.v:76: $finish called at 10020000 (1ps)
```

총 10,000개의 테스트 모두 **PASSED**로 통과하였다.

결론

과제를 진행하면서 발견한 궁금증을 강의 내용을 통해 해결할 수 있었다. Shift 연산의 경우, 연산의 대상, 즉 Shift를 할 대상이 `operand1`이 아닌 `operand2`가 되는 이유가 궁금했는데, R-type 명령어의 구조를 뜯어보면 알 수 있었다. 각각의 레지스터와 변수명은 `rs(operand1)`, `rt(operand2)`, `shamt(shift amount)`로 대응되는데, `rs`와 `rt`를 더하거나 빼는 ADD/SUB 연산과 달리 shift 연산의 경우 `rs`를 사용하지 않고 `rt`에 shift의 대상을 저장하기 때문이다.

그 외에는 사실 간단한 연산 기능 구현이라 큰 어려움은 없었던 것 같다.